

**NATIONAL UNIVERSITY OF SINGAPORE**  
**DEPARTMENT OF ELECTRICAL & COMPUTER ENGINEERING**

**ACADEMIC YEAR 2025-2026**  
**SEMESTER 1**

**EE2213: Introduction to AI**

**TUTORIAL 2: L2-L3**

---

**Question 1** (Lecture 2)

Which of the following statements about tree search and graph search are correct? Select all that apply.

- (a) If a state is reachable via multiple paths, tree search will create two separate nodes corresponding to each path
- (b) Tree search checks if a node has been visited before adding it to the search queue
- (c) Tree search is more efficient than graph search
- (d) Graph search checks if a node has been visited before adding it to the search queue

**Ans:** (a), (d)

**Question 2** (Lecture 3)

Which of the following is true about greedy best-first search? Select all that apply.

- (a) It guarantees both completeness and optimality when using a well-designed heuristic
- (b) It expands nodes based solely on the lowest cumulative path cost from the start state
- (c) It expands the node that appears closest to the goal, based only on the heuristic function
- (d) It behaves identically to uniform-cost search when edge costs are equal

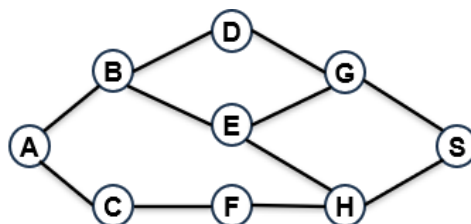
**Ans:** (c)

**Question 3** (Lecture 2)

You are given the following undirected graph.

- (i) Write a Python program to perform a Breadth-First Search (BFS) traversal starting from node 'A'. Print the order in which nodes are visited.
- (ii) Write a Python program to perform a Depth-First Search (DFS) traversal starting from node 'A'. Print the order in which nodes are visited.

Note: The order of nodes visited may differ depending on the order neighbors are processed.



**Ans:** Please refer to TUT2-Q3-bfs.py and TUT2-Q3-dfs.py for sample implementations.

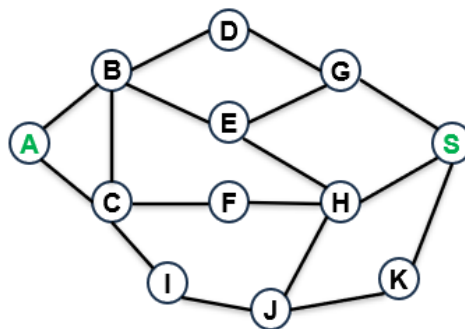
- (i) One possible sequence could be ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'S']. Note: The exact sequence returned may vary depending on the order in which neighboring nodes are explored during BFS.
- (ii) One possible sequence could be ['A', 'C', 'F', 'H', 'S', 'G', 'E', 'B', 'D']. Note: The exact sequence returned may vary depending on the order in which neighboring nodes are explored during DFS.

#### Question 4 (Lecture 2)

Your friend is flying from Arequipa (A), a historic city and the second-largest in Peru, to Singapore (S). However, there is no direct flight between the two cities.

You decide to plan a route with the minimum number of flight segments needed for your friend. To do this, you model the available flight connections as an undirected graph as shown below, where nodes represent cities, and edges represent direct flights between cities. The modeled graph is shown below.

- (i) Which strategy is better for finding the fewest number of flight segments: Breadth-First Search (BFS) or Depth-First Search (DFS)?
- (ii) Write a Python programme to compute the minimum number of flight segments from Arequipa to Singapore.



**Ans:**

- (i) To find the fewest number of flight segments from Arequipa (A) to Singapore (S), the most suitable strategy is Breadth-First Search (BFS).

BFS explores level by level, meaning it first visits all cities that are one flight away, then two flights away, and so on.

The first time BFS reaches the destination (S), it guarantees the path has the minimum number of edges (i.e., flight segments).

DFS, on the other hand, may go deep into one path without checking if there's a shorter one available. It does not guarantee the shortest path in terms of the number of edges.

- (ii) Please refer to TUT2-Q4.py as a sample implementation.

One possible path with the fewest flight transfers from Arequipa to Singapore: ['A',

'B', 'D', 'G', 'S']

Minimum number of flight segments: 4

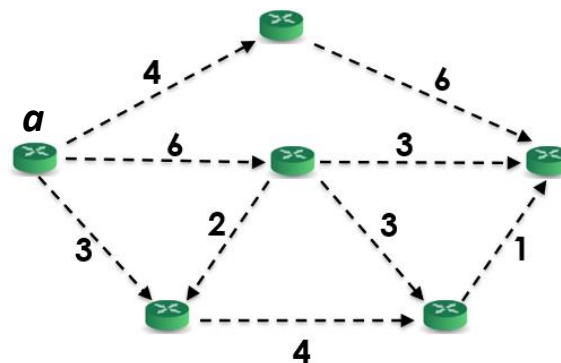
Note: Multiple equally optimal solutions may exist: There can be more than one path with the same minimum number of flight segments. Since in our program, BFS stops when it finds the first shortest path, it returns just one of these equally valid solutions.

### Question 5 (Lecture 3)

Signal Propagation in a Network (Adapted from LeetCode #743)

You are given a communication network consisting of several nodes connected by directed links, each with an associated transmission delay (indicated by the numbers beside each edge), as illustrated in the graph below. Given a starting node *a* (the leftmost node), determine the minimum time required for a signal to reach all nodes in the network.

- (i) Which search algorithm that you have learned so far would be most suitable for this problem?
- (ii) Implement your solution in Python using the selected search algorithm. Your program should:
  - Model the network as a graph.
  - Compute the minimum time it takes for the signal to reach every node.



**Ans:** (i) The most suitable algorithm is Dijkstra's Algorithm.

Justification: Dijkstra's algorithm is specifically designed to find the shortest paths from a single source node to all other nodes in a graph with non-negative edge weights. It works by greedily selecting the unvisited node with the smallest known distance from the source, ensuring that the first time it reaches a node, it has found the shortest path to it.

(ii) Please refer to TUT2-Q5.py as a sample implementation. Below is one possible set of output:

Shortest Distance to a : 0

Shortest Path to a : ['a']

Shortest Distance to b : 4

Shortest Path to b : ['a', 'b']

Shortest Distance to c : 6

Shortest Path to c : ['a', 'c']

Shortest Distance to d : 3

Shortest Path to d : ['a', 'd']

Shortest Distance to e : 7

Shortest Path to e : ['a', 'd', 'e']

Shortest Distance to f : 8

Shortest Path to f : ['a', 'd', 'e', 'f']

Note: There may be more than one shortest path with the same total distance — Dijkstra's algorithm returns just one of them.